# Final Design Document

## Landing Solutions

For Panther Cloud Air

**Team Members:**
Jack Jaramillo, Kamryn Wagner, Finley Myers, Sean Harder, Jimi Adegoroye

# Table of Contents

**Final Design Document** 1
**Table of Contents** 2
**Table of Tables** 3
**Connections** 4
**Major Algorithms** 6
The Great Circle Algorithm 6
Demand Model & Scoring Calculations 6
Heading Calculations 6
Flight Time Calculations 7
Flight Cost Calculations 7
Ticket Price Cost Calculations 7
**Module Signatures [Backend]** 8
**Module Signatures [Frontend]** 12
**Work Breakdown** 15
Finley 15
Kamryn 15
Jimi 15
Sean 15
Jack 15

# Table of Tables

**Table #1** ..... 8
**Table #2** ..... 8
**Table #3** ..... 9
**Table #4** ..... 9
**Table #5** ..... 10
**Table #6** ..... 10
**Table #7** ..... 11
**Table #8** ..... 11
**Table #9** ..... 13
**Table #10** ..... 13
**Table #11** ..... 14
**Table #12** ..... 14
**Table #13** ..... 15
**Table #14** ..... 15

# Connections

**Name**: User Login Authentication

**Purpose**: This connection allows users to enter login credentials and verifies them against the data stored in the database before granting them access to the system

**Data**: Input - username and password entered by the user; Output - authentication to the system with successful login

**Processing constraints**: The login request must be completed before the user can access anything from our system and be validated by our backend. The system also has to wait for a response before redirecting a user

**Protocol / transmission rules**: This uses HTTP POST request and the JSON format is used to send data (username, password) in a request-response model. Each login attempt is also processed individually

**Other filtering/processing**: The frontend validates empty fields before sending any requests while that backend performs case-sensitive credential matching. The plain text is also hashed and we are comparing hashed passwords for validation.

**Name**: Bulk Flight Data Retrieval

**Purpose**: This loads the complete flight schedule from the database into the frontend, allowing all flight queries to be handled locally without repeated database connections

**Data**: Input - no input, happens on initialization Output - full dataset from the all_flights table

**Processing constraints**: This must occur before any flight search or schedule display is used and the full dataset must be successfully retrieved. The data must also be consistent with the database at the time of the initial lode.

**Protocol / transmission rules**: This uses HTTP GET requests with query parameters and the results returned in a JSON format. It follows a one time request response and eliminates repeated database calls.

**Other filtering/processing**: Stores the dataset in local storage after retrieval and any other operations are performed locally.

**Name**: Initial Airport Data Retrieval

**Purpose**: Loads all airport data into the frontend at application startup

**Data**: Input - no user input; Output - full dataset from the airport table in the database

**Processing constraints**: Must be completed before the user interacts with the system and the data must be accurate, consistent and static during the use of the system.

**Protocol / transmission rules**: This uses HTTP GET request with the simulation passed as a query parameter and transmitted in JSON format. It follows a one time request response and eliminates repeated database calls.

**Other filtering/processing**: Stores the dataset in local storage after retrieval and any other operations or filters are performed locally.

# Major Algorithms

## The Great Circle Algorithm

The Great Circle Algorithm was used to find the distance between any two airports based on their GPS Coordinates. This algorithm was designed to return the shortest possible direct path between any 2 points in the world. In the code, we had predefined each airport with their Latitude and Longitude coordinates. Then, each time we simulated a flight, we took the departing airports coordinates and the arriving airports coordinates and passed them into a pre-defined function. This function would further calculate the distance by converting the coordinates from degrees to radians. After this, it determined the sine and cosine of both latitude coordinates along with the cosine of the difference between the longitudes of the destination airport and the starting airport. In the end, the 2 latitudes were multiplied and added together, and multiplied with the cosine of the longitudes. This final number would be multiplied by the radius of the earth, in miles, which gave us the final distance.

## Demand Model & Scoring Calculations

The demand model was used to estimate passenger demand between different metropolitan areas and act as our route scheduler. The model takes the population of the origin city and the relative population of all possible destination cities as ‘inputs’. Using the population of the origin city, it multiplies it by the given travel rate and market share as well as the population of the destination city divided by population of all other destinations. This determines how many passengers are wanting to fly between each city and assigns a specific route value to Panther Cloud Air. We then score the route by determining how many miles that route is required to travel, because if it's less miles, then its less flight time, which means we can run that route multiple times.

## Heading Calculations

In order to determine the heading of the simulated flight, we created a function that would return a value of degree headings. It first takes the longitude and latitude of 2 airports and turns them into radians. Next it does a calculation very similar to the Great Circle Algorithm, but instead determines 2 values. The first value is the sine of the difference of 2 longitudes times the cosine of the destination airport. The second value is the cosine of the departing airports latitude times the sine of the destination airports latitude. Both these values are subtracted by the cosine of the difference of the 2 longitudes. Once these calculations are done, the final calculation is finding the arctangent and turning it back into degrees which then gives the final heading in degrees.

## Flight Time Calculations

In order to determine the total flight time of any simulated flight, we created a function that would return the total flight time, in minutes, based on pre-set rules given to us by the client. The first calculation we make is to figure out the taxing time on the tarmac which is based on the population at the departing airport. Then we add 1 minute to allow for the plane to take off from the runway. The next 3 calculations, climb time, flying time, and descent time, are based on the altitude of the flight which is determined based on the distance. Each of these times are calculated based on specific rules based on the altitude, but the flying time also uses the distance, speed, and heading of the plane. Once the simulation has landed, we add another 2 minutes for the plane to land and slow down on the arrival runway. Then we add another taxi time based on the arrival population and a "turn around" time based on if the plane needed fuel or not when it lands. All these numbers are dynamically added together and returned as a single count in minutes.

## Flight Cost Calculations

We created a function that returns the total cost so that we can determine the total operating cost of a simulated flight in US dollars. This is based on the airports that are involved and the fuel that is consumed. The function takes the total gallons of fuel needed for the flight and takes into account whether or not it is international in the sense of if it is involved with Paris. Calculation 1 determines the fuel cost. If the flight is international and arriving at Paris, the function converts the gallons into liters by multiplying by 3.78541, then multiplies by the Paris fuel price of 1.97 euros per liter, and finally converts the euro total into US dollars using our exchange rate. If the flight is domestic, it simply multiplies the gallons by the US fuel price of 6.19 dollars per gallon. Once the fuel cost is calculated, the function adds the fees for the airport. For the departing airport, it checks if the origin is Paris or a US airport and charges either 2100 euros or 2000 dollars for an operation. Each takeoff and each landing is one operation each so a single flight is 2 operations and the check is also done for the arriving airport. All of these costs are put together into one variable and returned in US dollars as the total operating cost.

## Ticket Price Cost Calculations

Determining the price for a flight that is simulated meant that we created a function that returns what the price per seat is based on what the operating costs of the flight and also the total number of seats on the aircraft. Since flights are only profitable if only 30% of seats on the plane were filled, tickets are priced at that as the load factor which means the flight is profitable even if only 31% of the seats are filled. This is strategic because if we hit our goals of 75% load, then every flight we do not have issues with making profit. The function calculates the break-even passenger count by multiplying the total seats by 30% representing the load factor as mentioned earlier. The reason we price at 30 percent instead of the actual target of 75 percent is risk

management because if only that amount of seats sell, then we break even. This is strategic even if risky and conservative because it ensures that the airline does not lose money even on the planes that seem to underperform. Revenue is calculated by multiplying the number of passengers by the ticket price and the profit is made by subtracting the total operating cost from what is made from the revenue.

# Module Signatures [Backend]

## Table #1

| | |
|---|---|
| **Function Name** | simulateCurrentDay() |
| **Function ID** | FlightSimulation_001 |
| **Functions Description** | The purpose of this function is to fully simulate the current day of flights. It will filter through all flights created by our demand model, pick only the ones that match the current day selected, and simulate their flights. Throughout the flights, it will update each plane's status as it is used and place them into maintenance when needed. |
| **Input Parameters** | None |
| **Output Info** | It sends each simulated flight and its data, plus the plane's data, to the database to be stored for access. |

## Table #2

| | |
|---|---|
| **Function Name** | airport.distanceTo() |
| **Function ID** | FlightSimulation_002 |
| **Functions Description** | The purpose of this function is to determine the distance between 2 airports. It calculates the distance by having the 2 |

| | |
|---|---|
| | airports GPS coordinates and uses the Great Circle Algorithm for the calculation |
| **Input Parameters** | Is called locally by the starting airport object and passes in the ending airport object. This gives it access to both airports GPS coordinates |
| **Output Info** | Returns the distance as a unit of miles |

Table #3

| | |
|---|---|
| **Function Name** | flight.hdg() |
| **Function ID** | FlightSimulation_003 |
| **Functions Description** | The purpose of this function is to determine the heading of the flight that the plane will take. It calculates the heading through a modified version of the Great Circle Algorithm. |
| **Input Parameters** | The function takes 2 sets of coordinates as 4 parameters:<br>**Lat1** (integer) - latitude of the starting airport<br>**Lon1** (integer) - longitude of the starting airport<br>**Lat2** (integer) - latitude of the destination airport<br>**Lon2** (integer) - longitude of the destination airport |
| **Output Info** | Returns the heading as a unit of degrees |

Table #4

| | |
|---|---|
| **Function Name** | flight.flightTime() |
| **Function ID** | FlightSimulation_004 |
| **Functions Description** | The the purpose of this function is to calculate the total flight time based on rules set by the clients papers |
| **Input Parameters** | The function takes 9 parameters:<br>**Mi** (double) - total miles of the flight<br>**Spd** (integer) - max speed of the plane<br>**Hdg** (integer) - degree heading of the flight |

| | |
|---|---|
| | **Intl** (boolean) - if the flight is international or not<br>**oPop** (integer) - population of the starting airport<br>**oHub** (boolean) - if the starting airport is a hub or not<br>**dPop** (integer) - population of the destination airport<br>**dHub** (boolean) - if the destination airport is a hub or not<br>**needFuel** (boolean) - if the plane needs to be refueled on landing |
| **Output Info** | Returns the total flight time as a unit of minutes |

Table #5

| | |
|---|---|
| **Function Name** | flightCost() - jimi |
| **Function ID** | FlightSimulation_005 |
| **Functions Description** | Calculates the total operating cost of a single flight leg in US dollars and adds up the fuel cost and airport fees based on whether the flight is domestic or international. If the destination is Paris, fuel is priced in euros per liter and converted to USD. Airport fees are charged per operation at either US or Paris rates depending on the origin and destination. |
| **Input Parameters** | ● gal (double) - total gallons of fuel needed for the flight<br>● intl (boolean) - true if the flight is international<br>● from_eu (boolean) - true if the departing airport is Paris CDG<br>● to_eu (boolean) - true if the arriving airport is Paris CDG |
| **Output Info** | The output returns a double that is the total flight operating cost in US dollars as that includes the fuel cost and the one takeoff fee and landing fee at the destination. The value is passed on ticketPrice() to determine per-seat pricing. |

Table #6

| | |
|---|---|
| **Function Name** | ticketPrice() - jimi |
| **Function ID** | FlightSimulation_006 |
| **Functions Description** | Determining the price for a flight that is simulated meant that we created a function that returns what the price per seat is based on what the operating costs of the flight and also the total number of seats on the aircraft. Since |

| | |
|---|---|
| | flights are only profitable if only 30% of seats on the plane were filled, tickets are priced at that as the load factor which means the flight is profitable. Revenue is calculated by multiplying the number of passengers by the ticket price and the profit is made by subtracting the total operating cost from what is made from the revenue. |
| **Input Parameters** | op_cost (double) - the total operating cost of the flight in USD, returned from flightCost()<br>seats (int) - total passenger capacity of the aircraft |
| **Output Info** | The output returns the double representing the ticket price per seat in dollars. If the seats are multiplied by 0.30 ends up equaling 0, then the function returns 0 to prevent a divide by zero error. The value is used by the revenue function to then calculate what the total ticket sales is. |

Table #7

| | |
|---|---|
| **Function Name** | fleetLocations() |
| **Function ID** | FlightSimulation_007 |
| **Functions Description** | This function will return the entire fleet's locations separated by their location. If a plane is currently in maintenance, it will show they are unavailable and where their current location is. |
| **Input Parameters** | None |
| **Output Info** | This function returns no value, but will update the database with all current locations of the planes in order to see where they are. |

Table #8

| | |
|---|---|
| **Function Name** | increaseDay() |
| **Function ID** | FlightSimulation_008 |
| **Functions Description** | This function is responsible for increasing the simulated day by 1 value to the next day. It will prevent any admin user from increasing the day past 14. |
| **Input Parameters** | None |

| | |
|---|---|
| **Output Info** | Returns no value, only updates the currentDay variable to increase by 1. |

# Module Signatures [Frontend]

## Table #9

| | |
|---|---|
| **Function Name** | OnLaunchScripts |
| **Function ID** | WebFunction-001 |
| **Functions Description** | The purpose of this function is to run once and only once on successful launch of the app. This allows the team to easily add additional functionality, and manage data. Inside the OnLaunchScripts function, the team can call a series of other functions, add or remove data to local storage, as well as add or change state data. |
| **Input Parameters** | None |
| **Output Info** | None |

## Table #10

| | |
|---|---|
| **Function Name** | GetAllFlights |
| **Function ID** | WebFunction-002 |
| **Functions Description** | The purpose of this function is to successfully GET request our database and return all created flights. |
| **Input Parameters** | None |
| **Output Info** | The response gets parsed with .json and is saved as a **Flight** which is a TypeScript interface type that matches our AllFlights schema. Types can be found in **types.d.ts** |

Table #11

| | |
|---|---|
| **Function Name** | ProtectedRoute |
| **Function ID** | WebFunction-003 |
| **Functions Description** | The purpose of this function/component is to make sure users are logged in, authenticated, and have the correct role to be seeing the content they do. It checks the pathname, and their user obj to check if they all are good otherwise they redirect them back. |
| **Input Parameters** | **user**: This is just the user obj, it has id, their role, and isAuthenticated in the obj.<br>**correctRole**: This is passed in via the component, we use this to check against the user's role to make sure they match.<br>**Children:** This allows the ProtectedRoute component to be wrapped around the actual targeted page for react to render. |
| **Output Info** | None |

Table #12

| | |
|---|---|
| **Function Name** | handleLogin |
| **Function ID** | WebFunction-004 |
| **Functions Description** | The purpose of this function is to handle the login process when the user presses the actual login button. It will clear any errors made and then call validate() to check the input fields, stopping if validation fails. If validation succeeds, it sets the loading state and sends a POST request to the backend endpoint with the username and password in JSON format for authentication. |
| **Input Parameters** | **Username:** Uses this field to gather request data<br>**Password:** Uses this field to gather request data |
| **Output Info** | Page redirection on success, error if failure |

Table #13

| | |
|---|---|
| **Function Name** | validate |
| **Function ID** | WebFunction-005 |
| **Functions Description** | The purpose of this function is to validate the login form input before submission to the database. It trims the username and password values of excess spaces and checks whether both fields are filled in. If both are empty, or if either one is missing, it throws an error message and returns false. If both inputs are present, it returns true so processing can continue. |
| **Input Parameters** | **Username:** Uses the username field to validate if each field contains input after trimming spaces, and it uses setError to send the error message when input is invalid.<br>**Password:** Uses the password field to validate if each field contains input after trimming spaces, and it uses setError to send the error message when input is invalid. |
| **Output Info** | Boolean that outputs false when fails and true when both username and password fields are valid. |

Table #14

| | |
|---|---|
| **Function Name** | resetTimer |
| **Function ID** | WebFunction-006 |
| **Functions Description** | The purpose of this function is to reset the user inactivity timer. It checks localStorage for an existing timeout stored under the inactivity_timeout status, and if one is found, it will clear the save to avoid multiple inactivity timers running at the same time. This helps ensure proper session timeout handling in the application. |
| **Input Parameters** | The function does not take any explicit parameters as input. It just uses localStorage to read the value into the function and determine if it should be cleared or not. |
| **Output Info** | None |

# Work Breakdown

## Finley

Major Algorithms - Great Circle, Heading, and Flight Time calculations
Module Signatures [Backend] - simluateCurrentDay() airport.distanceTo(), flight.hdg(), flightTime(), fleetLocations(), increaseDay()

## Kamryn

Module signatures [Frontend] - handleLogin, validate, resetTimer

## Jimi

Major Algorithms - Flight Cost and Ticket Price Cost calculations
Module Signatures [Backend] - flightCost() - jimi, ticketPrice() - jimi

## Sean

Module signatures [Frontend]

## Jack

Database Connections